We'll examine a simple but practical exercise to demonstrate the effectiveness of the conversational pattern. For the next exercise, we'll employ two assistants in an ABT that converse over the same thread. The next listing shows the tree's construction in code with the respective assistants.

Listing 6.10 `agentic_conversation_btree.py`

```
root = py_trees.composites.Sequence("RootSequence", memory=True)
bug_file = """
# code not shown
"""

thread = api.create_thread()     #1

debug_code = create_assistant_action_on_thread(     #2
    thread=thread,
    action_name="Debug code",
    assistant_name="Python Debugger",
    assistant_instructions=textwrap.dedent(f"""
    Here is the code with bugs in it:
    {bug_file}
    Run the code to identify the bugs and fix them.
    Be sure to test the code to ensure it runs without errors or throws
any exceptions.
    """),
)
root.add_child(debug_code)

verify = create_assistant_condition_on_thread(     #3
    thread=thread,
    condition_name="Verify",
    assistant_name="Python Coding Assistant",
    assistant_instructions=textwrap.dedent(
        """
    Verify the solution fixes the bug and there are no more issues.
    Verify that no exceptions are thrown when the code is run.
    Reply with SUCCESS if the solution is correct, otherwise return FAILURE.
    If you are happy with the solution, save the code to a file called
fixed_bug.py.
    """,
    ),
)
root.add_child(verify)
tree = py_trees.trees.BehaviourTree(root)
while True:
    tree.tick()
    if root.status == py_trees.common.Status.SUCCESS:
        break    #4
    time.sleep(20)
```

#1 Creates a message thread for the assistants to share and converse over
#2 Creates the debug code action with a special assistant
#3 Creates the verification condition to test if the code is fixed or not
#4 The tree will continue to run until the root sequence completes with success.